Queries Explanation + Output

1-SELECT FirstName, LastName FROM Instructors WHERE Specialization = 'Mathematics';

	FirstName	LastName
1	Jane	Smith

Fetches the first and last names of instructors who specialize in Mathematics.

2-SELECT FirstName, LastName FROM Students.

	FirstName	LastName
1	Alice	Smith
2	Bob	Johnson
3	Charlie	Brown
4	David	Lee
5	Emma	Jones

Retrieves the first and last names of all students.

3-SELECT * FROM Students WHERE DateOfBirth > '2004-01-01'.

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	alice.smith@gmail.com	71501442	2005-05-14
2	2	Bob	Johnson	bob.johnson@gmail.com	03815618	2004-07-23
3	3	Charlie	Brown	charlie.brown@gmail.com	71289988	2005-09-12
4	4	David	Lee	david.lee@gmail.com	81619153	2004-01-15

Selects all columns for students born after January 1, 2004.

The * is a wildcard that means "all columns."

The WHERE clause is used to filter rows based on a condition.

4-SELECT * FROM Students ORDER BY LastName DESC.

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	alice.smith@gmail.com	71501442	2005-05-14
2	4	David	Lee	david.lee@gmail.com	81619153	2004-01-15
3	5	Emma	Jones	emma.jones@gmail....	78892328	2003-11-03
4	2	Bob	Johnson	bob.johnson@gmail....	03815618	2004-07-23
5	3	Charlie	Brown	charlie.brown@gmail....	71289988	2005-09-12

Retrieves all student details sorted by last name in descending order.

The ORDER BY clause is used to sort the rows in the result set.

5-SELECT COUNT(*) AS TotalStudents FROM Students.

	TotalStudents
1	5

Returns the total number of students in the students' table.

COUNT is an aggregate function used to count rows in a table or the result of a query.

AS TotalStudents: Assigns the result a column name, in this case, TotalStudents.

AS gives a name to the column or table in the query result to make it easier to read

6-UPDATE Students SET Email = 'new.email@example.com' WHERE StudentID = 1.

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	new.email@example.com	71501442	2005-05-14

Updates the email address of the student with StudentID = 1 to new.email@example.com.

SET: Specifies the new value for a column in an UPDATE statement.

7-SELECT FirstName, LastName

```
FROM Students
WHERE NOT EXISTS (
  SELECT CourseID
  FROM Courses
  WHERE NOT EXISTS (
    SELECT *
    FROM Enrollments
    WHERE Enrollments.CourseID = Courses.CourseID AND Enrollments.StudentID = Students.StudentID
  )
);
```

	FirstName	LastName

Fetch students who are enrolled in all available courses.

The outer query selects students.

The first NOT EXISTS checks if there is any course the student is not enrolled in.

The second, NOT EXISTS confirms whether a specific student is enrolled in each course.

8-SELECT StudentID FROM Enrollments WHERE CourseID = 1

UNION

SELECT StudentID FROM Enrollments WHERE CourseID = 2;

	StudentID
1	1
2	2

Get a list of students enrolled in either Course 1 or Course 2.

UNION merges two queries, removing duplicates by default.

Fetches StudentID for students enrolled in Course 1 and Course 2.

9-SELECT Students.FirstName, Students.LastName, Courses.CourseName

FROM Students

JOIN Enrollments ON Students.StudentID = Enrollments.StudentID

JOIN Courses ON Enrollments.CourseID = Courses.CourseID;

	FirstName	LastName	CourseName
1	Alice	Smith	Introduction to Data Science
2	Alice	Smith	Advanced Mathematics
3	Bob	Johnson	Advanced Mathematics
4	Charlie	Brown	Organic Chemistry
5	David	Lee	English001
6	Emma	Jones	Economics for Beginners

Get each student's first and last name along with the courses they are enrolled in.

JOIN connects Students, Enrollments, and Courses tables.

This returns the course name for each student.

10-SELECT Specialization, AVG(Salary) AS AvgSalary

FROM Instructors

GROUP BY Specialization;

	Specialization	AvgSalary
1	Biochemistry	5700.000000
2	Data Science	7000.000000
3	Economics	2900.000000
4	Language	4500.000000
5	Mathematics	4800.000000

Find the average salary for each specialization.

GROUP BY groups the instructors by Specialization.

AVG(Salary) calculates the average salary for each group.

11-SELECT Specialization, AVG(Salary) AS AvgSalary

FROM Instructors

GROUP BY Specialization

HAVING AVG(Salary) > 5000;

	Specialization	AvgSalary
1	Biochemistry	5700.000000
2	Data Science	7000.000000

Same as the previous query but filters groups where the average salary exceeds 5000.

The HAVING clause filters groups based on aggregate functions like AVG.

12-SELECT * FROM Students WHERE Email IS NULL;

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth

Retrieve all students without an email.

WHERE Email IS NULL checks for missing values in the Email column.

13-UPDATE Students

SET PhoneNumber = '00000000'

WHERE PhoneNumber = '71501442';

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	new.email@example.com	00000000	2005-05-14

Set a default phone number for student whose number is 71501442

Updates the PhoneNumber column only for rows where it's currently 71501442.

14- UPDATE Instructors

SET Salary = 7500.00

WHERE InstructorID = 1;

	InstructorID	FirstName	LastName	Email	PhoneNumber	Specialization	Salary
1	1	John	Doe	john.doe@gmail.com	76444333	Data Science	7500.00

Update the salary of the instructor with InstructorID = 1.

Targets only the specific instructor by InstructorID.

15- SELECT * FROM Courses WHERE Price > 1000;

	CourseID	CourseName	Description	Duration	Price
1	1	Introduction to Data Science	Learn the basics of data science, including tools a...	15	1500.00
2	2	Advanced Mathematics	An advanced course on calculus and linear algebra.	10	1600.00
3	3	Organic Chemistry	Organic Chemistry studies carbon-based compou...	15	2700.00
4	5	Economics for Beginners	An introductory course on micro and macroecono...	13	1550.00

Find courses priced above 1000.

WHERE Price > 1000 filters for expensive courses.

16-SELECT TOP 1 * FROM Courses

ORDER BY Price DESC;

	CourseID	CourseName	Description	Duration	Price
1	3	Organic Chemistry	Organic Chemistry studies carbon-based compound...	15	2700.00

Retrieve the single most expensive course.

TOP 1 limits the result to one row.

ORDER BY Price DESC sorts courses by price in descending order.

17- SELECT AVG(Price) AS AvgPrice FROM Courses;

	AvgPrice
1	1660.000000

Calculate the average price of all courses.

Uses the AVG function to compute the average of the Price column

18- SELECT * FROM Classrooms WHERE Capacity > 30;

	ClassroomID	RoomNumber	Capacity
1	3	C303	35
2	4	D404	40
3	5	E505	35

Find all classrooms with a capacity greater than 30.

Filters rows where Capacity > 30.

19- SELECT CASE

```
        WHEN Capacity < =20 THEN 'Small'
        WHEN Capacity BETWEEN 21 AND 40 THEN 'Medium'
        ELSE 'Large'
    END AS CapacityCategory,
    COUNT(*) AS RoomCount
FROM Classrooms
GROUP BY CASE
        WHEN Capacity < =20 THEN 'Small'
        WHEN Capacity BETWEEN 21 AND 40 THEN 'Medium'
        ELSE 'Large'
    END;
```

	CapacityCategory	RoomCount
1	Medium	5

Categorize classrooms by size and count how many belong to each category.

CASE creates size categories.

COUNT(*) counts classrooms in each category.

GROUP BY ensures rows are grouped by category.

20- SELECT Days, COUNT(*) AS ScheduleCount

FROM Schedules

GROUP BY Days;

	Days	ScheduleCount
1	Friday	1
2	Monday, Tuesday	2
3	Monday, Wedn...	1
4	Thursday	2

Count how many schedules exist for each day.

Groups schedules by the Days column.

Counts the number of schedules for each day.

21- SELECT MaterialType, COUNT(*) AS MaterialCount

FROM Materials

GROUP BY MaterialType;

	MaterialType	MaterialCount
1	Audio, Powerpoint	1
2	Lab,Worksheets	1
3	PDF	2
4	Powerpoint	1

Count how many materials exist for each type.

Groups materials by MaterialType.

Counts the number of materials in each type.

22- SELECT * FROM Feedbacks WHERE Rating >= 4;

	FeedbackID	StudentID	CourseID	Rating	Comments	DateSubmitted
1	1	1	1	5	Excellent course and explanation with great exa...	2024-01-20
2	2	1	2	4	Very detailed explanations, but a bit slow-paced.	2024-01-21
3	3	2	2	4	The instructor was excellent, although the materia...	2024-01-16
4	6	5	5	4	Good content and explanation.	2024-01-19

Retrieve feedback with ratings of 4 or higher.

Filters rows with a Rating of 4 or above.

23- SELECT CourseID, AVG(Rating) AS AvgRating

FROM Feedbacks

GROUP BY CourseID;

	CourseID	AvgRating
1	1	5
2	2	4
3	3	3
4	4	1
5	5	4

Calculate the average rating for each course.

Groups feedback by CourseID.

Calculates the average rating for each course.

24- SELECT * FROM Payments WHERE Status = 'Pending';

	PaymentID	CourseID	StudentID	Amount	PaymentDate	PaymentMethod	Status
1	6	2	1	1600.00	2024-01-10 10:00:00.000	Credit Card	Pending

Retrieve all payments with a Pending status.

Filters rows where the Status is 'Pending'.

25- SELECT PaymentMethod, SUM(Amount) AS TotalAmount

FROM Payments

GROUP BY PaymentMethod;

	PaymentMethod	TotalAmount
1	Bank Transfer	3150.00
2	Cash	2700.00
3	Credit Card	4050.00

Calculates the total payment amount for each payment method.

26- SELECT * FROM Assignments WHERE DueDate BETWEEN '2024-02-01' AND '2024-02-28';

	AssignmentID	AssignmentTitle	CourseID	Description	MaxGrade	DueDate
1	1	Data Analysis P33 and D10	1	Analyze the dataset after calculating the decile a...	100	2024-02-15
2	2	Calculus Problem Set	2	determine the union and the intersection and pre...	100	2024-02-20
3	3	Organic Chemistry Lab	3	Submit lab results and report.	100	2024-02-25

Retrieve assignments due within February 2024.

BETWEEN: Filters rows with dates in a specific range.

27- SELECT Students.FirstName, Students.LastName

FROM Enrollments

JOIN Students ON Enrollments.StudentID = Students.StudentID

WHERE CourseID = 2;

	FirstName	LastName
1	Alice	Smith
2	Bob	Johnson

Get names of students enrolled in course with CourseID = 2.

Join Enrollments and Students using StudentID.

WHERE CourseID = 2: Filters by a specific course.

28- SELECT CourseID, COUNT(StudentID) AS EnrollmentCount

FROM Enrollments

GROUP BY CourseID;

	CourseID	EnrollmentCount
1	1	1
2	2	2
3	3	1
4	4	1
5	5	1

Count students enrolled in each course.

COUNT(StudentID): Aggregates total enrollments per CourseID.

GROUP BY CourseID: Groups results by each course.

29- SELECT Status, COUNT(*) AS AttendanceCount

FROM Attendances

GROUP BY Status;

	Status	AttendanceCount
1	Absent	1
2	Present	5

Count attendance statuses (Present, Absent, etc.).

COUNT(*): Counts rows for each unique Status.

```
30- SELECT  s.FirstName,  s.LastName,  c.CourseName,  a.Status
FROM  Students s
JOIN Enrollments e ON s.StudentID = e.StudentID
JOIN Courses c ON e.CourseID = c.CourseID
JOIN Attendances a ON s.StudentID = a.StudentID
   AND e.CourseID = a.ScheduleID
WHERE a.Status IS NOT NULL;
```

	FirstName	LastName	CourseName	Status
1	Alice	Smith	Introduction to Data Science	Present
2	Bob	Johnson	Advanced Mathematics	Present

This query fetches the following information:

Student's First Name (s.FirstName) and Last Name (s.LastName) from the Students table.

Course Name (c.CourseName) that the student is enrolled in from the Courses table.

Attendance Status (a.Status) of the student for that course from the Attendances table.

```
31- SELECT *
FROM Courses
WHERE CourseName LIKE '%Chem%'
   OR LEN(CourseName) = 8
   OR CourseName LIKE 'Eco%';
```

	CourseID	CourseName	Description	Duration	Price
1	3	Organic Chemistry	Organic Chemistry studies carbon-based compound...	15	2700.00
2	5	Economics for Beginners	An introductory course on micro and macroeconomi...	13	1550.00

-- Contains "Chem"

-- Exactly 8 letters

-- Starts with "Eco"

Find courses meeting specific naming conditions.

LIKE: Filters substrings (% for any characters).

LEN(CourseName) = 8: Filters names with exactly 8 characters.

33- SELECT * FROM Students ORDER BY DateOfBirth ASC;

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	5	Emma	Jones	emma.jones@gmail.com	78892328	2003-11-03
2	4	David	Lee	david.lee@gmail.com	81619153	2004-01-15
3	2	Bob	Johnson	bob.johnson@gmail.com	03815618	2004-07-23
4	1	Alice	Smith	new.email@example.c...	00000000	2005-05-14
5	3	Charlie	Brown	charlie.brown@gmail.c...	71289988	2005-09-12

List students by age (youngest to oldest).

ORDER BY DateOfBirth ASC: Sorts chronologically.

34- SELECT c.CourseName

FROM Schedules s

JOIN Courses c ON s.CourseID = c.CourseID

WHERE s.Days = 'Monday, Tuesday';

	CourseName
1	Advanced Mathematics
2	Advanced Mathematics

Retrieve courses scheduled on Mondays and Tuesdays.

Join Schedules and Courses using CourseID.

WHERE Days: Filters specific days.

35- SELECT DISTINCT Specialization

FROM Instructors;

	Specialization
1	Biochemistry
2	Data Science
3	Economics
4	Language
5	Mathematics

This query retrieves a list of all unique specializations from the instructor's table. The DISTINCT keyword ensures that only unique values are returned. If there are multiple instructors with the same specialization, it will only appear once in the result set.

36- SELECT CourseName

FROM Courses

INTERSECT

SELECT CourseName

FROM Feedbacks f

JOIN Courses c ON f.CourseID = c.CourseID;

	CourseName
1	Advanced Mathematics
2	Economics for Beginners
3	English001
4	Introduction to Data Sc...
5	Organic Chemistry

This query finds the names of courses that appear in both the Courses table and the Feedbacks table.

The INTERSECT operator returns the common rows between the two queries.

The first query gets all CourseName entries from the Courses table.

The second query joins the Feedback table with the Courses table (using CourseID) and retrieves the course names that have feedback.

The result will show the courses that are both listed in the Courses table and have feedback provided in the Feedback table.

37- SELECT *

FROM Students

WHERE PhoneNumber IS NULL.

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth

This query selects all students from the students table who do not have a phone number listed.

The IS NULL condition checks for rows where the PhoneNumber column contains no value (i.e., it is NULL).

38- SELECT AVG(Salary) AS AverageSalary, MAX(Salary) AS HighestSalary, MIN(Salary) AS LowestSalary

FROM Instructors;

	AverageSalary	HighestSalary	LowestSalary
1	5080.000000	7500.00	2900.00

This query calculates and returns the average, maximum, and minimum salaries from the Instructors table.

AVG(Salary): Finds the average salary of all instructors.

MAX(Salary): Finds the highest salary among instructors.

MIN(Salary): Finds the lowest salary among instructors.

39- SELECT CourseID, COUNT(StudentID) AS EnrolledStudents

FROM Enrollments

GROUP BY CourseID

HAVING COUNT(StudentID) > 1;

	CourseID	EnrolledStudents
1	2	2

This query finds all courses that have more than one student enrolled.

GROUP BY CourseID: Groups the records by CourseID so that we can aggregate data by course.

COUNT(StudentID): Counts the number of students enrolled in each course.

HAVING COUNT(StudentID) > 1: Filters the courses to include only those with more than one student enrolled.

40- SELECT *

FROM Students

WHERE StudentID IN (

  SELECT StudentID

  FROM Feedbacks

  WHERE Rating = 5

);

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	new.email@example.com	00000000	2005-05-14

This query returns all student records for students who have given a rating of 5 in the Feedbacks table.

The subquery (SELECT StudentID FROM Feedback WHERE Rating = 5) selects the StudentID of all students who have given a rating of 5.

The main query uses IN to select all records from the students table where the StudentID matches any of those in the subquery.

41- SELECT CourseName

FROM Courses

WHERE Price < ALL (SELECT Salary FROM Instructors);

	CourseName
1	Introduction to Data Science
2	Advanced Mathematics
3	Organic Chemistry
4	English001
5	Economics for Beginners

This query retrieves all course names where the course price is lower than every instructor's salary.

ALL compares the Price of the course against all salaries in the instructors table.

Only courses with a price less than all instructor salaries will be included in the result.

42- SELECT *

FROM Students s

WHERE EXISTS (

  SELECT 1

  FROM Enrollments e

  WHERE e.StudentID = s.StudentID

);

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	new.email@example.com	00000000	2005-05-14
2	2	Bob	Johnson	bob.johnson@gmail.com	03815618	2004-07-23
3	3	Charlie	Brown	charlie.brown@gmail.com	71289988	2005-09-12
4	4	David	Lee	david.lee@gmail.com	81619153	2004-01-15
5	5	Emma	Jones	emma.jones@gmail.com	78892328	2003-11-03

This query returns all students who are enrolled in at least one course.

The subquery checks for the existence of any enrollment record (SELECT 1) for each student.

EXISTS returns TRUE if the subquery finds any record for that student in the Enrollments table, meaning the student is enrolled in at least one course

43-SELECT PhoneNumber, COUNT(*)

FROM Students

GROUP BY PhoneNumber

HAVING COUNT(*) > 1;

	PhoneNumber	(No column name)

This query identifies phone numbers that are associated with multiple students.

44- SELECT CourseName,

(SELECT AVG(Price) FROM Courses) AS AveragePrice

FROM Courses;

	CourseName	AveragePrice
1	Introduction to Data Science	1660.000000
2	Advanced Mathematics	1660.000000
3	Organic Chemistry	1660.000000
4	English001	1660.000000
5	Economics for Beginners	1660.000000

This query retrieves the name of each course along with the average price of all courses.

45- SELECT FirstName, LastName,

CASE

WHEN DateOfBirth < '2004-01-01' THEN 'Adult'

ELSE 'Minor'

END AS AgeGroup

FROM Students;

	FirstName	LastName	AgeGroup
1	Alice	Smith	Minor
2	Bob	Johnson	Minor
3	Charlie	Brown	Minor
4	David	Lee	Minor
5	Emma	Jones	Adult

This query classifies students as either "Adult" or "Minor" based on their date of birth.

The CASE statement checks if the DateOfBirth is earlier than '2004-01-01' (i.e., the student is 20 years or older).

If the condition is true, the student is classified as an "Adult"; otherwise, they are classified as a "Minor".

46- INSERT INTO Students (FirstName, LastName, DateOfBirth, PhoneNumber)

VALUES ('Glinda', Blue, '2005-03-15', '81899999);

	StudentID	FirstName	LastName	Email	PhoneNumber	DateOfBirth
1	1	Alice	Smith	new.email@example.com	00000000	2005-05-14
2	2	Bob	Johnson	bob.johnson@gmail.com	03815618	2004-07-23
3	3	Charlie	Brown	charlie.brown@gmail.com	71289988	2005-09-12
4	4	David	Lee	david.lee@gmail.com	81619153	2004-01-15
5	5	Emma	Jones	emma.jones@gmail.com	78892328	2003-11-03
6	6	Glinda	Blue	NULL	81899999	2005-03-15

Adding a student into the Students table where her email is null .

47- SELECT FirstName, LastName

FROM Students s

WHERE NOT EXISTS (

  SELECT 1

  FROM Enrollments e

  WHERE e.StudentID = s.StudentID

);

	FirstName	LastName
1	Glinda	Blue

This query returns the first name and last name of students who are not enrolled in any course.

48- WITH CoursePopularity AS (

  SELECT CourseID, COUNT(StudentID) AS EnrolledStudents

  FROM Enrollments

  GROUP BY CourseID

)

SELECT c.CourseName, cp.EnrolledStudents

FROM Courses c

```
JOIN CoursePopularity cp ON c.CourseID = cp.CourseID

WHERE cp.EnrolledStudents >= 2;
```

	CourseName	EnrolledStudents
1	Advanced Mathematics	2

This query finds courses with more than 2 students enrolled and shows their names along with the number of enrolled students.

The WITH clause creates a temporary result set (CoursePopularity) that counts the number of students enrolled in each course.

The main query joins the Courses table with this temporary result set and filters the courses where more than 2 students are enrolled.

```
49- SELECT CourseName

FROM Courses

WHERE Price < SOME (SELECT Salary FROM Instructors);
```

	CourseName
1	Introduction to Data Science
2	Advanced Mathematics
3	Organic Chemistry
4	English001
5	Economics for Beginners

This query retrieves some course names where the course price is lower than every instructor's salary.

SOME compares the Price of the course against SOME salaries in the instructors table.

```
50- SELECT Courses.CourseName, Instructors.FirstName, Instructors.LastName

FROM Courses

INNER JOIN Courses_Instructors ON Courses.CourseID = Courses_Instructors.CourseID

INNER JOIN Instructors ON Courses_Instructors.InstructorID = Instructors.InstructorID;
```

	CourseName	FirstName	LastName
1	Introduction to Data Science	John	Doe
2	Advanced Mathematics	Jane	Smith
3	Organic Chemistry	Emily	Davis
4	English001	Michael	Brown
5	Economics for Beginners	Olivia	Wilson
6	Advanced Mathematics	John	Doe

Retrieve a list of courses and their instructors.

INNER JOIN connects tables by matching CourseID and InstructorID.

It ensures only rows with matches in all tables are included.

51- SELECT Courses.CourseName, Instructors.FirstName

FROM Courses

LEFT OUTER JOIN Courses_Instructors ON Courses.CourseID = Courses_Instructors.CourseID

LEFT OUTER JOIN Instructors ON Courses_Instructors.InstructorID = Instructors.InstructorID;

	CourseName	FirstName
1	Introduction to Data Science	John
2	Advanced Mathematics	Jane
3	Advanced Mathematics	John
4	Organic Chemistry	Emily
5	English001	Michael
6	Economics for Beginners	Olivia

Retrieve all courses, including those without instructors.

LEFT OUTER JOIN: Includes all courses, even if they don't match in Courses_Instructors or Instructors.

Rows without matches will have NULL in instructor fields.

52- SELECT Courses.CourseName, Instructors.FirstName

FROM Courses

RIGHT OUTER JOIN Courses_Instructors ON Courses.CourseID = Courses_Instructors.CourseID

RIGHT OUTER JOIN Instructors ON Courses_Instructors.InstructorID = Instructors.InstructorID;

	CourseName	FirstName
1	Introduction to Data Science	John
2	Advanced Mathematics	John
3	Advanced Mathematics	Jane
4	Organic Chemistry	Emily
5	English001	Michael
6	Economics for Beginners	Olivia

List all instructors, even if they aren't teaching any courses.

RIGHT OUTER JOIN: Includes all instructors, even without course matches.

Rows without matches will have NULL in course fields.

53- SELECT Courses.CourseName, Instructors.FirstName

FROM Courses

FULL OUTER JOIN Courses_Instructors ON Courses.CourseID = Courses_Instructors.CourseID

FULL OUTER JOIN Instructors ON Courses_Instructors.InstructorID = Instructors.InstructorID;

	CourseName	FirstName
1	Introduction to Data Science	John
2	Advanced Mathematics	Jane
3	Advanced Mathematics	John
4	Organic Chemistry	Emily
5	English001	Michael
6	Economics for Beginners	Olivia

Retrieve all courses and instructors, regardless of matches.

Combines results of LEFT OUTER JOIN and RIGHT OUTER JOIN.

Unmatched rows from both sides will have NULL fields.

54- SELECT Students.FirstName, Students.LastName, Courses.CourseName,

Instructors.FirstName AS InstructorFirstName, Instructors.LastName AS InstructorLastName

FROM Students

INNER JOIN Enrollments ON Students.StudentID = Enrollments.StudentID

INNER JOIN Courses ON Enrollments.CourseID = Courses.CourseID

INNER JOIN Courses_Instructors ON Courses.CourseID = Courses_Instructors.CourseID

INNER JOIN Instructors ON Courses_Instructors.InstructorID = Instructors.InstructorID;

	FirstName	LastName	CourseName	InstructorFirstName	InstructorLastName
1	Alice	Smith	Introduction to Data Science	John	Doe
2	Alice	Smith	Advanced Mathematics	Jane	Smith
3	Alice	Smith	Advanced Mathematics	John	Doe
4	Bob	Johnson	Advanced Mathematics	Jane	Smith
5	Bob	Johnson	Advanced Mathematics	John	Doe
6	Charlie	Brown	Organic Chemistry	Emily	Davis
7	David	Lee	English001	Michael	Brown
8	Emma	Jones	Economics for Beginners	Olivia	Wilson

Show students, their courses, and instructors.

Links five tables: Students, Enrollments, Courses, Courses_Instructors, Instructors.

Uses INNER JOIN to ensure all relationships are valid.

55- SELECT S1.FirstName AS Student1, S1.LastName AS LastName1,

S2.FirstName AS Student2, S2.LastName AS LastName2,

Courses.CourseName

FROM Students S1

JOIN Enrollments E1 ON S1.StudentID = E1.StudentID

JOIN Enrollments E2 ON E1.CourseID = E2.CourseID

JOIN Students S2 ON E2.StudentID = S2.StudentID

JOIN Courses ON E1.CourseID = Courses.CourseID

WHERE S1.StudentID < S2.StudentID;

	Student1	LastName1	Student2	LastName2	CourseName
1	Alice	Smith	Bob	Johnson	Advanced Mathematics

Identify pairs of students in the same course.

Self-join Students table to compare one student (S1) with another (S2).

WHERE S1.StudentID < S2.StudentID avoids duplicate pairs.

56- SELECT *

FROM Courses

WHERE CourseName = 'Introduction toData Science';

	CourseID	CourseName	Description	Duration	Price
1	1	Introduction to Data Science	Learn the basics of data science, including too...	15	1500.00

Retrieve details for the "Introduction to Data Science" course.

Uses WHERE to filter rows.

57- ALTER TABLE Submissions

ADD CONSTRAINT CHK_Grade CHECK (Grade >= 0 AND Grade <= 100);

Add a rule ensuring grades are valid.

ALTER TABLE: Modify table structure.

CHECK: Enforces a logical condition on column values.

58- SELECT Students.FirstName, Students.LastName,

COUNT(DISTINCT Enrollments.CourseID) AS NumberOfCourses,

AVG(Submissions.Grade) AS AverageGrade

FROM Students

JOIN Enrollments ON Students.StudentID = Enrollments.StudentID

JOIN Submissions ON Students.StudentID = Submissions.StudentID

GROUP BY Students.StudentID, Students.FirstName, Students.LastName

HAVING COUNT(DISTINCT Enrollments.CourseID) > 1

ORDER BY AverageGrade DESC;

	FirstName	LastName	NumberOfCourses	AverageGrade
1	Alice	Smith	2	91

Find students with multiple courses and calculate their average grade.

COUNT: Counts unique courses.

AVG: Computes the average grade.

HAVING: Filters groups based on aggregate functions.

59- SELECT Instructors.FirstName, Instructors.LastName,

SUM(Payments.Amount) AS TotalPayments

FROM Instructors

JOIN Courses_Instructors ON Instructors.InstructorID = Courses_Instructors.InstructorID

JOIN Courses ON Courses_Instructors.CourseID = Courses.CourseID

JOIN Payments ON Payments.CourseID = Courses.CourseID

GROUP BY Instructors.InstructorID, Instructors.FirstName, Instructors.LastName

ORDER BY TotalPayments DESC;

	FirstName	LastName	TotalPayments
1	John	Doe	4700.00
2	Jane	Smith	3200.00
3	Emily	Davis	2700.00
4	Olivia	Wilson	1550.00
5	Michael	Brown	950.00

Calculate the total payments received by each instructor.

Links tables: Instructors, Courses_Instructors, Courses, and Payments.

SUM(Payments.Amount): Aggregates total payment for each instructor.

GROUP BY: Groups results by instructor.

ORDER BY TotalPayments DESC: Sorts instructors in descending order of payments.

```
60- SELECT Students.FirstName, Students.LastName,
      COUNT(Attendances.AttendanceID) AS MissedClasses
FROM Students
JOIN Attendances ON Students.StudentID = Attendances.StudentID
WHERE Attendances.Status = 'Absent'
GROUP BY Students.StudentID, Students.FirstName, Students.LastName
HAVING COUNT(Attendances.AttendanceID) > 1;
```

FirstName	LastName	MissedClasses

Find students who have missed more than one class.

Links Students and Attendances tables.

Filters rows where Status = 'Absent'.

HAVING COUNT(Attendances.AttendanceID) > 1: Ensures the count of absences exceeds

```
61- SELECT TOP 1 Courses.CourseName,
      COUNT(Enrollments.StudentID) AS EnrollmentCount
FROM Courses
JOIN Enrollments ON Courses.CourseID = Enrollments.CourseID
GROUP BY Courses.CourseName
ORDER BY EnrollmentCount DESC;
```

	CourseName	EnrollmentCount
1	Advanced Mathematics	2

Identify the course with the highest number of enrolled students.

COUNT(Enrollments.StudentID): Counts the number of students per course.

GROUP BY Courses.CourseName: Groups rows by course.

ORDER BY EnrollmentCount DESC: Sorts courses by enrollment count in descending order.

TOP 1: Returns only the top course.

62-SELECT Rating, Comments FROM Feedbacks WHERE CourseID = 1;

	Rating	Comments
1	5	Excellent course and explanation with great exa...

Retrieve the ratings and comments for course with CourseID = 1.

Feedbacks table: Contains ratings and comments for courses.

WHERE CourseID = 1: Filters results to show only feedback for the course with ID 1.

63- SELECT RoomNumber, Capacity FROM Classrooms WHERE Capacity > 30;

	RoomNumber	Capacity
1	C303	35
2	D404	40
3	E505	35

List classrooms that have a capacity greater than 30.

Classrooms table: Contains classroom details.

WHERE Capacity > 30: Filters classrooms to only those with capacity greater than 30.

64- SELECT s.FirstName, s.LastName, SUM(p.Amount) AS TotalPaid,

RANK() OVER (ORDER BY SUM(p.Amount) DESC) AS Rank

FROM Payments p

JOIN Students s ON p.StudentID = s.StudentID

GROUP BY s.StudentID, s.FirstName, s.LastName;

	FirstName	LastName	TotalPaid	Rank
1	Alice	Smith	3100.00	1
2	Charlie	Brown	2700.00	2
3	Bob	Johnson	1600.00	3
4	Emma	Jones	1550.00	4
5	David	Lee	950.00	5

Rank students based on the total amount paid.

SUM(p.Amount): Calculates the total payment per student.

RANK() OVER (ORDER BY SUM(p.Amount) DESC): Ranks students based on total payment in descending order.

JOIN Payments and Students: Links the Payments table with Students table.

65- INSERT INTO Students (FirstName, LastName, Email, PhoneNumber, DateOfBirth)

VALUES

('Sophia', 'Taylor', 'sophia.taylor@gmail.com', '81325667', '2004-04-10'),

('Liam', 'Anderson', 'liam.anderson@gmail.com', '76543889', '2003-12-20');

Insert two new students into the Students table.

INSERT INTO Students: Adds new records to the Students table.

The VALUES keyword inserts the values for FirstName, LastName, Email, PhoneNumber, and DateOfBirth for both students.

66- SELECT * FROM STUDENTS;

Retrieve all columns and rows from the STUDENTS table.

SELECT *: Selects all columns in the STUDENTS table.

67- INSERT INTO Enrollments (StudentID, CourseID)

VALUES

(6, 3), -- Assuming Sophia is the 6th StudentID

(7, 3); -- Assuming Liam is the 7th StudentID

Insert enrollment records for Sophia and Liam in course 3.

INSERT INTO Enrollments: Adds records to the Enrollments table.

VALUES (6, 3) and (7, 3): Specifies the StudentID and CourseID for both students.

68- SELECT * FROM ENROLLMENTS;

Retrieve all columns and rows from the ENROLLMENTS table.

SELECT *: Selects all columns in the ENROLLMENTS table.

69- INSERT INTO Feedbacks (StudentID, CourseID, Rating, Comments, DateSubmitted)

VALUES

(6, 3, 5, 'Excellent coverage of the subject.', '2024-02-01'),

(7, 3, 4, 'Good course but a bit challenging.', '2024-02-02');

Insert feedback for students in course 3.

INSERT INTO Feedbacks: Adds records to the Feedbacks table.

VALUES: Provides the StudentID, CourseID, Rating, Comments, and DateSubmitted for both students.

70- SELECT * FROM FEEDBACKS;

Retrieve all columns and rows from the FEEDBACKS table.

SELECT *: Selects all columns in the FEEDBACKS table.

71- SELECT

```
    c.CourseName,
    c.Description,
    CONCAT(i.FirstName, ' ', i.LastName) AS InstructorName
FROM Courses c
JOIN Courses_Instructors ci ON c.CourseID = ci.CourseID
JOIN Instructors i ON ci.InstructorID = i.InstructorID;
```

	CourseName	Description	InstructorName
1	Introduction to Data Science	Learn the basics of data science, including tools an...	John Doe
2	Advanced Mathematics	An advanced course on calculus and linear algebra.	Jane Smith
3	Organic Chemistry	Organic Chemistry studies carbon-based compound...	Emily Davis
4	English001	An English course develops reading, writing, speaki...	Michael Brown
5	Economics for Beginners	An introductory course on micro and macroeconomi...	Olivia Wilson
6	Advanced Mathematics	An advanced course on calculus and linear algebra.	John Doe

Retrieve course names, descriptions, and the corresponding instructor's name.

JOIN Courses and Instructors: Combines the Courses, Courses_Instructors, and Instructors tables.

CONCAT(i.FirstName, ' ', i.LastName): Combines the first and last name of the instructor into one column.

72- SELECT

```
    c.CourseName,
    COUNT(e.StudentID) AS TotalStudents
FROM Courses c
LEFT JOIN Enrollments e ON c.CourseID = e.CourseID
GROUP BY c.CourseName
ORDER BY TotalStudents DESC;
```

	CourseName	TotalStudents
1	Organic Chemistry	3
2	Advanced Mathematics	2
3	Economics for Beginners	1
4	English001	1
5	Introduction to Data Science	1

Find the total number of students enrolled in each course.

COUNT(e.StudentID): Counts the number of students for each course.

LEFT JOIN: Ensures all courses are included, even those with no enrollments.

ORDER BY TotalStudents DESC: Sorts the courses by the total number of students in descending order.

73- SELECT TOP 3

```
    s.FirstName,
    s.LastName,
    a.AssignmentTitle,
    sub.Grade
FROM Submissions sub
JOIN Students s ON sub.StudentID = s.StudentID
JOIN Assignments a ON sub.AssignmentID = a.AssignmentID
ORDER BY sub.Grade DESC;
```

	FirstName	LastName	AssignmentTitle	Grade
1	Alice	Smith	Data Analysis P33 and D10	95
2	Bob	Johnson	Calculus Problem Set	92
3	Alice	Smith	Calculus Problem Set	88

Retrieve the top 3 students with the highest grades.

TOP 3: Limits the result to the top 3 highest grades.

ORDER BY sub.Grade DESC: Sorts the results in descending order of grade.

74- SELECT

```
    RoomNumber,
    Capacity
FROM Classrooms
WHERE Capacity >= 25 AND ClassroomID NOT IN (
    SELECT DISTINCT ClassroomID
    FROM Schedules
);
```

RoomNumber	Capacity

Find classrooms with capacity greater than 25 that are not used in any schedule.

NOT IN: Filters out classrooms that appear in the Schedules table.

WHERE Capacity >= 25: Only includes classrooms with a capacity of 25 or more.

75- SELECT

```
    s.FirstName,
    s.LastName,
    p.PaymentMethod,
    p.Status,
    p.Amount
FROM Payments p
JOIN Students s ON p.StudentID = s.StudentID
```

```
JOIN Courses c ON p.CourseID = c.CourseID
WHERE c.CourseName = 'Introduction to Data Science';
```

	FirstName	LastName	PaymentMethod	Status	Amount
1	Alice	Smith	Credit Card	Paid	1500.00

Retrieve payment details for the course "Introduction to Data Science".

JOIN Payments, Students, and Courses: Combines the three tables.

WHERE c.CourseName = 'Introduction to Data Science': Filters the results to only show payments for this specific course.

```
76- SELECT
  FirstName,
  LastName,
  Specialization,
  Salary
FROM Instructors
WHERE Salary > 5000;
```

	FirstName	LastName	Specialization	Salary
1	John	Doe	Data Science	7500.00
2	Emily	Davis	Biochemistry	5700.00

Retrieve instructors with a salary greater than 5000.

WHERE Salary > 5000: Filters the instructors by salary.

```
77- WITH InstructorEarnings AS (
  SELECT
    i.InstructorID,
    i.FirstName,
    i.LastName,
    i.Salary,
    SUM(c.Price) AS TotalCourseFees
```

```
    FROM Instructors i
    INNER JOIN Courses_Instructors ci ON i.InstructorID = ci.InstructorID
    INNER JOIN Courses c ON ci.CourseID = c.CourseID
    GROUP BY i.InstructorID, i.FirstName, i.LastName, i.Salary
)
SELECT TOP 3
    FirstName,
    LastName,
    Salary,
    TotalCourseFees,
    (Salary + TotalCourseFees) AS TotalEarnings
FROM InstructorEarnings
ORDER BY TotalEarnings DESC;
```

	FirstName	LastName	Salary	TotalCourseFees	TotalEarnings
1	John	Doe	7500.00	3100.00	10600.00
2	Emily	Davis	5700.00	2700.00	8400.00
3	Jane	Smith	4800.00	1600.00	6400.00

Find the top 3 instructors based on their total earnings (salary + course fees).

WITH InstructorEarnings AS (...): Creates a temporary result set that calculates instructor earnings.

SELECT TOP 3: Limits the result to the top 3 instructors by total earnings.

```
78- WITH ClassroomUtilization AS (
    SELECT
        cl.ClassroomID,
        cl.RoomNumber,
        cl.Capacity,
        COUNT(s.ClassroomID) AS UtilizedSeats
    FROM Classrooms cl
    LEFT JOIN Schedules s ON cl.ClassroomID = s.ClassroomID
    GROUP BY cl.ClassroomID, cl.RoomNumber, cl.Capacity
```

```
)
SELECT
    RoomNumber,
    Capacity,
    UtilizedSeats,
    (UtilizedSeats * 100.0 / Capacity) AS UtilizationPercentage
FROM ClassroomUtilization
WHERE UtilizedSeats * 100.0 / Capacity < 50;
```

	RoomNumber	Capacity	UtilizedSeats	UtilizationPercentage
1	A101	30	2	6.666666666666
2	B202	25	1	4.000000000000
3	C303	35	1	2.857142857142
4	D404	40	1	2.500000000000
5	E505	35	1	2.857142857142

Find classrooms that are utilized less than 50% based on their seating capacity.

WITH ClassroomUtilization AS (...): Calculates classroom utilization.

WHERE UtilizedSeats * 100.0 / Capacity < 50: Filters classrooms where utilization is less than 50%.

```
79- CREATE TABLE EnrollmentLogs (
    LogID INT PRIMARY KEY IDENTITY(1,1),
    StudentID INT NOT NULL,
    CourseID INT NOT NULL,
    EnrollmentDate DATETIME DEFAULT GETDATE()
);

CREATE TRIGGER LogEnrollment
ON Enrollments
AFTER INSERT
AS
```

```
BEGIN

   INSERT INTO EnrollmentLogs (StudentID, CourseID, EnrollmentDate)

  SELECT StudentID, CourseID, GETDATE()
   FROM INSERTED;
END;

INSERT INTO Enrollments (StudentID, CourseID)
VALUES (1, 1); -- Replace with valid StudentID and CourseID values

SELECT * FROM EnrollmentLogs;
```

	LogID	StudentID	CourseID	EnrollmentDate
1	1	1	1	2024-12-12 22:59:41.820

AFTER INSERT Trigger: Fires automatically when a new row is inserted into the Enrollments table.

Temporary Table (INSERTED):

Contains the data that was just added to the Enrollments table.

Extract the StudentID and CourseID from the INSERTED table.

Insert the values, along with the current date (GETDATE()), into the EnrollmentLogs table.

LogEnrollment: Records when a student enrolls in a course.

```
80- CREATE TABLE PaymentStatusLog (
   LogID INT PRIMARY KEY IDENTITY(1,1),
   PaymentID INT NOT NULL,
   OldStatus VARCHAR(50),
   NewStatus VARCHAR(50),
   ChangeDate DATETIME DEFAULT GETDATE()
);
```

```
CREATE TRIGGER LogPaymentStatusChange
ON Payments
AFTER UPDATE
AS
BEGIN

    INSERT INTO PaymentStatusLog (PaymentID, OldStatus, NewStatus)
    SELECT d.PaymentID, d.Status AS OldStatus, i.Status AS NewStatus
    FROM DELETED d
    JOIN INSERTED i ON d.PaymentID = i.PaymentID
    WHERE d.Status <> i.Status; -- Only log changes where the status has been updated
END;

INSERT INTO Payments (Status, Amount, StudentID, CourseID)
VALUES ('Pending', 100.00, 1, 1);

UPDATE Payments
SET Status = 'Completed'
WHERE PaymentID = 1;

SELECT * FROM PaymentStatusLog;
```

	LogID	PaymentID	OldStatus	NewStatus	ChangeDate
1	1	1	Paid	Completed	2024-12-12 22:58:25.427

Fires automatically when the Payments table is updated.

Temporary Tables (INSERTED and DELETED):

DELETED: Contains the data before the update.

INSERTED: Contains the data after the update.

Compare the Status column in the DELETED and INSERTED tables.

If the status has changed (d.Status <> i.Status), log the change in the PaymentStatusLog table.

Both triggers automate logging and reduce the need for manual record-keeping.